

```

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload one image
uploaded = files.upload()

# Get image name
image_name = list(uploaded.keys())[0]

# Read image in grayscale
img = cv2.imread(image_name, 0)

# Create noisy image
noise = np.random.normal(0, 30, img.shape)

noisy_img = img.astype(np.float32) + noise
noisy_img = np.clip(noisy_img, 0, 255).astype(np.uint8)

# Preprocess noisy image using Gaussian Blur
filtered_img = cv2.GaussianBlur(noisy_img, (5, 5), 0)

# Create ORB feature detector
orb = cv2.ORB_create(nfeatures=500)

# Detect features and descriptors
kp1, des1 = orb.detectAndCompute(img, None)
kp2, des2 = orb.detectAndCompute(noisy_img, None)
kp3, des3 = orb.detectAndCompute(filtered_img, None)

# Create Brute Force Matcher
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

# Match original with noisy image
matches_noisy = bf.match(des1, des2)
matches_noisy = sorted(matches_noisy, key=lambda x: x.distance)

# Match original with filtered image
matches_filtered = bf.match(des1, des3)
matches_filtered = sorted(matches_filtered, key=lambda x: x.distance)

# Draw best matches
match_noisy_img = cv2.drawMatches(
    img, kp1,
    noisy_img, kp2,
    matches_noisy[:30],
    None,
    flags=2
)

match_filtered_img = cv2.drawMatches(
    img, kp1,
    filtered_img, kp3,
    matches_filtered[:30],
    None,
    flags=2
)

# Display results
plt.figure(figsize=(15, 10))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")

```

```
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(noisy_img, cmap="gray")
plt.title("Noisy Image")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(match_noisy_img)
plt.title("Feature Matching with Noise")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(match_filtered_img)
plt.title("Feature Matching after Preprocessing")
plt.axis("off")

plt.tight_layout()
plt.show()

# Print results
print("Matches with noise:", len(matches_noisy))
print("Matches after preprocessing:", len(matches_filtered))
```

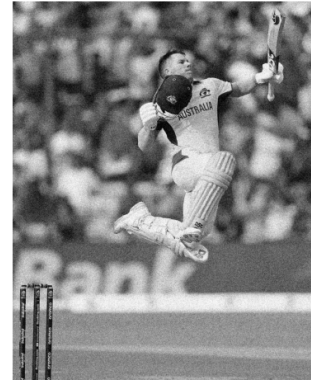
[Choose files](#) images (4).jpg

images (4).jpg(image/jpeg) - 118594 bytes, last modified: 21/07/2026 - 100% done
 Saving images (4).jpg to images (4).jpg

Original Image



Noisy Image



Feature Matching with Noise



Feature Matching after Preprocessing



Matches with noise: 367

Matches after preprocessing: 298

